ID: 222311069

Question 1+7: Explain how gradient Descent optimi-zation works. Describe how the algorithm uses the "negative of the gradient" to reach the minimum point of the loss curve.

Ans:- Gradient Descent is an optimization algorithm that minimizes a model's error by adjusting its weights step by step. Think of it like standing on a hill blindfolded "feel the slope and keep stepping downhill until reach the lowest point.

The Negative Gradient:
The gradient at any point shows which direction is uphill (increasing loss). The algorithm moves in the opposite direction, the negative of the gradient, to reduce the loss. This process repeats until the minimum is reached.

new weight = old weight - (Learning Rate x gradient)

- Learning Rate: controls the step size (too high = overshoot, too low = very slow)
- Gradient: the slope of the loss curve at the current point.

Question – 2: Explain why the sigmoid activation function is widely used for probability prediction. Mention its range, the shape of its gradient, and one of its main drawbacks.

Ans: Sigmoid converts any input into a value between 0 and 1, making it deal for probability prediction (e.g., is this spam? 0.9 = 90% likely spam)

Formula and Range

$$\text{sigma}(x) = \frac{1}{(1 + e^{x})}$$

Range : 0 to 1

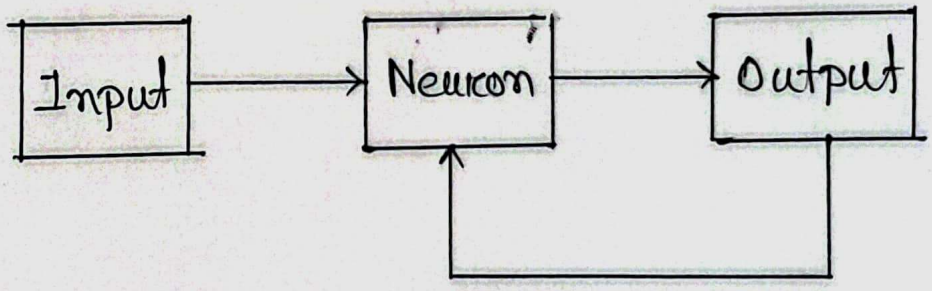
Gradient shape:
The gradient is bell-shapped highest near $x = 0$ and nearly zero at the extremes.

Main Drawback: Vanishing Gradient:
When inputs are fare from zero, the gradient approaches zero. In deep networks, this tiny. Gradient gets multiplied across many layers, causing early layers to stop learning. This is why ReLU has replaced sigmoid in most hidden layers.

Question - 3: Explain what a feedback Network is with an appropriate diagram. Why is it described as a "non-linear dynamic system.

Ans:- A Feedback Network (Recurrent Neural Network) is a neural network where the output is fed back as input to the same or previous layer, creating a loop. Unlike feed-forward networks, information flows in cycles.

Diagram:



Why it is a Non-Linear Dynamic System:

- Non-linear: Neurons use non-linear activation functions, so the input-output relationship is not a straight line.
- Dynamic: Output depends on current input AND past states. The network has memory that changes over time.

Question - 4: Contrast Machine Learning and Traditional Programming.

Ans:- Traditional programming requires a human to write explicit rules. Machine learning lets the computer discover those rules from data on its own.

Feature	Traditional programming	Machine Learning
How it works	Human writes rule manually	Computer learns rules from data
Input Vs output	Rules + Data = Output	Data + output = model
Flexibility	Only handles planned scenarios	handles new, unseen situations
Example	"win prize" in email = spam	Learns spam pattern automatically.

Question - 5:

Ans:-

	Actual Spam	Actual non spam	Total
Predicted spam	850 (TP)	50 (FP)	900
Predicted non spam	150 (FN)	1950 (TN)	2100
Total	1000	2000	3000

Calculations:-

$$\text{Accuracy} = \frac{TP + TN}{\text{Total}} = \frac{850 + 1950}{3000} = 93.33\%$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{850}{900} = 94.44\%$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{850}{1000} = 85.00\%$$

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{1950}{2000} = 97.50\%$$

Question-6: Contrast supervised learning based on three criteria: knowledge of output the type of dataset used, and the level of accuracy.

Ans:-

Criteria	Supervised Learning	Unsupervised Learning
knowledge of Output	Output labels are known during training	No labels model finds patterns on its.
Type of Dataset	Labeled dataset (e.g. spam/not spam)	Unlabeled dataset (e.g., raw customer data)
Level of Accuracy	Higher accuracy with clear guidance	Lower accuracy, no reference to compare.

Examples: supervised: spam detection, image recognition. ~~Unsupus~~ Unsupervised: customer grouping, anomaly detection.

Question - 8 : Write the mathematical formula for calculating the net input (y_{in}) and the final output (Y) of a neuron.

Ans: Net Input (y_in)

The neuron multiples each input by its weight, adds them all up, then adds a bias:

$$y\_in = b + (x1 * w1) + (w2 * w2) + \cdots (xn * wn)$$

$x1 \ldots xn$ = input values, $w1 \ldots wn$ = weights, b = bias

Final output (Y)

The net input is passed through an activation function to get the final result :

$$y = f(y\,in)$$

Using sigmoid as the activation function :

~~$y = f(y\,in)$~~

$$y = \frac{1}{1 + e^{-y\_in}}$$

The activation function adds non-linearity so the neuron can learn complex patterns, not just straight lines.